

# Shopademia — Master Project Blueprint

Oct 5, 2026 · @Abid Hossain Tushar

## 1. Project Overview

Shopademia is one central e-commerce API: products are uploaded once in an admin panel, and every authorized website or app reads them through that API. No website or app ever talks to the database directly.

### Who uses it

| Client                      | Role                                                 | Talks to       |
|-----------------------------|------------------------------------------------------|----------------|
| Admin Panel                 | Staff manage products, categories, orders, customers | Shopademia API |
| Shopademia Storefront       | Customer website                                     | Shopademia API |
| Wonderful Collection        | Second customer website (separate brand)             | Shopademia API |
| Android apps / future sites | Customer apps                                        | Shopademia API |

### Tech stack (version 1)

| Layer               | Choice                                              | Why                                                                   |
|---------------------|-----------------------------------------------------|-----------------------------------------------------------------------|
| Runtime / framework | Node.js + Express.js                                | Simple, huge community, beginner friendly                             |
| Database            | MySQL (via <code>mysql2</code> , parameterized SQL) | Reliable relational data; no ORM keeps the first version easy to read |
| API style           | REST, versioned under <code>/api/v1/</code>         | Predictable URLs; v2 can be added without breaking old clients        |
| Config / secrets    | <code>dotenv</code> and a <code>.env</code> file    | Secrets never live in code                                            |
| Passwords           | <code>bcrypt</code> (cost 12)                       | Proven, one-way hashing                                               |
| Auth                | JWT access tokens (short-lived)                     | Stateless, works for web and Android                                  |

| Layer      | Choice                                 | Why                                             |
|------------|----------------------------------------|-------------------------------------------------|
| Validation | <code>zod</code>                       | One schema per request; rejects bad input early |
| Later      | Redis (cache), HTTPS via reverse proxy | Added only when the core is stable              |

**Guiding rule:** build the smallest correct version of each phase, verify it, then stop. Nothing in this document is implementation code; it is the plan the code will follow.

## 2. Business/Technical Goals

The business goal is to enter each product once and sell it on many storefronts; the technical goal is to do that securely without over-building.

### Business goals

- Add a product once; show it on Shopademia, Wonderful Collection and future sites.
- Let admins create, edit, publish, hide and archive products from one place.
- Let customers browse, register, keep a profile, fill a cart and place orders.
- Let admins process orders and see customer activity.
- Make adding a new store a configuration task, not a rebuild.

### Technical goals

- One API is the only door to the database.
- Every admin action needs authentication and the right role.
- Safe by default: validated input, parameterized SQL, no leaked errors or secrets.
- Simple first: plain Express + MySQL, no Docker, queues or microservices until needed.
- Testable and phase-by-phase: each phase has a checklist and a Definition of Done.

**Non-goals for version 1:** payment gateway integration, marketing automation, multi-currency, multi-language, recommendation engines, microservices.

## 4. Complete Folder Structure

One repository, four top-level parts. The backend is built first; the admin and storefront folders stay empty until their phases.

```
shopademia/
├── backend/
│   ├── src/
│   │   └── config/          # env loading, db pool, constants
```

```

|   |   | └─ routes/           # URL -> controller wiring (v1/ subfolder)
|   |   | └─ controllers/      # read request, call service, send response
|   |   | └─ services/         # business rules (pricing, stock, checkout)
|   |   | └─ repositories/     # ALL SQL lives here
|   |   | └─ middleware/       # auth, roles, store-resolver, rate-limit,
errors
|   |   | └─ validators/       # zod schemas per endpoint
|   |   | └─ utils/            # logger, hash helpers, response helpers
|   |   | └─ app.js            # builds the Express app (no listen)
|   |   | └─ server.js         # starts the server
|   |   └─ tests/              # unit/ and integration/
|   |   └─ uploads/            # product images (git-ignored)
|   |   └─ .env                # real secrets (git-ignored)
|   |   └─ .env.example        # safe template, committed
|   |   └─ .gitignore
|   |   └─ package.json
└─ admin/                      # admin panel (Phase 6)
└─ storefront/                 # customer site (Phase 8)
└─ database/
|   └─ migrations/             # numbered .sql files: 001_..., 002_...
|   └─ seeds/                  # sample/dev data only
└─ docs/                       # this blueprint, API notes, runbooks

```

## Rules for the structure

- A file lives in exactly one layer: routes never hold SQL, repositories never touch `req / res`.
- No duplicate folders; no new top-level folder without a phase that needs it.
- `.env`, `uploads/` and `node_modules/` are always git-ignored.

## 5. Backend Architecture

The backend is a layered monolith: one Node.js app, small single-purpose layers, so a beginner can follow a request from URL to database.

### Request journey

1. Request arrives at `/api/v1/...` over HTTPS.
2. Global middleware runs: security headers, CORS, JSON parsing with a size limit, request ID, rate limit.
3. Store resolver identifies which store is calling (see section 13).
4. Auth middleware checks the token; role middleware checks permission.
5. Validator checks body, query and params with a zod schema.
6. Controller calls a service; the service applies business rules and calls repositories.

7. Repository runs a parameterized SQL query and returns plain objects.

8. Controller sends a consistent JSON response; errors go to the central error handler.

| Layer      | Responsibility                                  | Must NOT do                       |
|------------|-------------------------------------------------|-----------------------------------|
| Route      | Map URL + method to middleware and controller   | Contain logic                     |
| Controller | Translate HTTP to a service call and back       | Write SQL or business rules       |
| Service    | Business rules, transactions, permissions logic | Know about <code>req / res</code> |
| Repository | The only place SQL is written                   | Decide business rules             |
| Middleware | Cross-cutting checks (auth, limits, errors)     | Fetch business data               |

**Standards:** JavaScript (CommonJS or ESM, picked once in Phase 1 and never mixed), `async/await`, one connection pool, no global mutable state, config read only from `config/`.

## 6. Frontend/Storefront Architecture

A storefront is a pure API client: it holds no database credentials and no business rules, only screens and calls to `/api/v1/`.

- **Technology (first version):** plain HTML, CSS and JavaScript using `fetch`, so it stays beginner friendly. A framework (React, Vue) can replace it later without touching the API.
- **Pages:** home, category listing, product list with search and pagination, product detail, cart, checkout, login/register, account and order history.
- **Configuration:** each storefront has one config file with its API base URL and its public store key. The store key identifies the site; it is not a password.
- **Tokens:** customer tokens are kept in memory or a secure cookie, never in the URL; never log them.
- **Safe rendering:** product text from the API is inserted as text, not raw HTML, to prevent XSS.
- **Errors:** show friendly messages; never show raw API error details.
- **Shared by all storefronts:** the same API contract. Each brand differs only in theme, logo, and which products its store has enabled.

**Never:** connect to MySQL, call admin endpoints, or store secrets in frontend code (anything in a browser is public).

## 7. Admin Panel Architecture

The admin panel is a separate private web app that calls only `/api/v1/admin/...` endpoints; it is the single place where products are created.

- **Hosting:** its own subdomain (for example `admin.shopademia.store`), never mixed into a public storefront.
- **Login:** email + password against the admin auth endpoint; short-lived access token; automatic logout when it expires.
- **Screens (built in order):** login, dashboard, categories, products (create/edit/publish/archive/images), stores, orders, customers, audit log.
- **Access:** screens and buttons are shown by role, but the real protection is always on the API; hiding a button is not security.
- **Safety features:** confirm dialogs on delete/archive; soft delete instead of hard delete; every change recorded in the audit log.
- **Optional hardening later:** restrict by IP/VPN, two-factor login.

**Roles that use it:** `super_admin`, `admin`, `catalog_manager`, `order_manager` (details in section 12).

## 8. MySQL Database Architecture

The schema starts with about 20 tables in InnoDB with `utf8mb4`, created through numbered SQL migration files so every change is repeatable and reviewable.

### Conventions

- Primary key: `id BIGINT UNSIGNED AUTO_INCREMENT`.
- Every table has `created_at` and `updated_at`.
- Money is stored as an integer in the smallest unit (for example paisa), never as a float.
- Products, categories and stores use soft delete (`deleted_at` or `status`), never silent hard delete.
- Foreign keys are enforced; columns used in filters and joins are indexed.
- Table names are plural `snake_case`; each store-scoped table carries `store_id`.

### Tables by area

| Area   | Tables                                            | Purpose                                   |
|--------|---------------------------------------------------|-------------------------------------------|
| Stores | <code>stores</code> , <code>store_api_keys</code> | Each website/app, plus its hashed API key |

| Area        | Tables                                                                                     | Purpose                                    |
|-------------|--------------------------------------------------------------------------------------------|--------------------------------------------|
| Admin users | admin_users, roles, admin_user_roles                                                       | Staff accounts and what they may do        |
| Customers   | customers, customer_addresses, customer_sessions                                           | Shoppers, addresses, refresh tokens        |
| Catalog     | categories, products, product_variants, product_images, product_categories, product_stores | Central catalog and per-store availability |
| Inventory   | inventory                                                                                  | Stock count per variant                    |
| Cart        | carts, cart_items                                                                          | Active carts per customer and store        |
| Orders      | orders, order_items, order_status_history                                                  | Placed orders with price snapshots         |
| Operations  | audit_logs                                                                                 | Who did what, when, from where             |

### Key columns (essentials only)

| Table            | Important columns                                                                                                    |
|------------------|----------------------------------------------------------------------------------------------------------------------|
| products         | id, name, slug (unique), description, status ( draft / published / archived ), base_price, created_by, deleted_at    |
| product_variants | id, product_id, sku (unique), name, price, is_active                                                                 |
| product_stores   | product_id, store_id, is_visible, price_override (nullable)                                                          |
| orders           | id, store_id, customer_id, order_number (unique), status, subtotal, shipping, total, shipping_address snapshot       |
| order_items      | id, order_id, variant_id, product_name, sku, unit_price, quantity (snapshots, so later edits do not rewrite history) |
| admin_users      | id, email (unique), password_hash, is_active, last_login_at                                                          |
| audit_logs       | id, admin_user_id, action, entity_type, entity_id, store_id, ip, created_at                                          |

**Database accounts:** the API connects with a dedicated MySQL user that has only the rights it needs (no root , no DROP / GRANT ). Migrations run with a separate, more privileged account.

## 9. Database Relationships

The central idea: products belong to the platform, and product\_stores decides which store sees which product.

| Relationship                | Type                                | Rule                                                 |
|-----------------------------|-------------------------------------|------------------------------------------------------|
| products → product_variants | one-to-many                         | A product has at least one variant (size/colour/SKU) |
| products ↔ categories       | many-to-many via product_categories | A product can sit in several categories              |
| categories → categories     | self-reference ( parent_id )        | Simple category tree                                 |

| Relationship                                                                 | Type                                           | Rule                                             |
|------------------------------------------------------------------------------|------------------------------------------------|--------------------------------------------------|
| <code>products</code> → <code>product_images</code>                          | one-to-many                                    | Ordered images with alt text                     |
| <code>products</code> ↔ <code>stores</code>                                  | many-to-many via <code>product_stores</code>   | Per-store visibility and optional price override |
| <code>product_variants</code> → <code>inventory</code>                       | one-to-one                                     | Stock per variant                                |
| <code>customers</code> → <code>customer_addresses</code>                     | one-to-many                                    | Several saved addresses                          |
| <code>customers</code> + <code>stores</code> → <code>carts</code>            | one cart per customer per store                | Cart is store-specific                           |
| <code>carts</code> → <code>cart_items</code> → <code>product_variants</code> | one-to-many                                    | Unique (cart, variant)                           |
| <code>stores</code> → <code>orders</code>                                    | one-to-many                                    | Every order belongs to one store                 |
| <code>customers</code> → <code>orders</code>                                 | one-to-many                                    | History per customer                             |
| <code>orders</code> → <code>order_items</code>                               | one-to-many                                    | Items copy name, SKU and price at purchase time  |
| <code>admin_users</code> ↔ <code>roles</code>                                | many-to-many via <code>admin_user_roles</code> | Role-based permissions                           |
| <code>admin_users</code> → <code>audit_logs</code>                           | one-to-many                                    | Every important admin action                     |

**Delete behaviour:** child rows that have no meaning alone ( `product_images` , `cart_items` ) cascade; anything tied to money or history ( `orders` , `order_items` , `audit_logs` ) is never cascaded or deleted by the API.

## 10. REST API Architecture

Three URL groups share one `/api/v1/` prefix: public, customer and admin. The group tells you who may call it.

| Group    | Prefix         | Who                         | Examples                                                                            |
|----------|----------------|-----------------------------|-------------------------------------------------------------------------------------|
| Public   | /api/v1/       | Any store (needs store key) | GET /health, GET /products, GET /products/:slug, GET /categories                    |
| Customer | /api/v1/       | Logged-in customer          | POST /auth/register, POST /auth/login, GET /customers/me, /cart, /orders            |
| Admin    | /api/v1/admin/ | Staff with a role           | /admin/products, /admin/categories, /admin/orders, /admin/stores, /admin/audit-logs |

## Conventions

- Plural nouns, lowercase, hyphens; HTTP verbs carry the action: GET read, POST create, PATCH update, DELETE remove/archive.
- Pagination on every list: ?page=1&limit=20 (max limit 100), plus sort and q where useful.
- Success shape: { "success": true, "data": ..., "meta": { ... } }.
- Error shape: { "success": false, "error": { "code": "VALIDATION\_ERROR", "message": "...", "details": [...] } }.
- Status codes: 200, 201, 204, 400, 401, 403, 404, 409, 422, 429, 500.
- Public responses expose only customer-safe fields (never cost price, internal notes, admin IDs).
- Breaking changes go to /api/v2/ ; /api/v1/ keeps working.
- Every response carries a request ID for debugging.

## 11. Authentication & Authorization

Authentication asks "who are you?"; authorization asks "what may you do?". Both are checked on the API for every protected request.

### Three kinds of caller

| Caller              | Proves identity with | Notes                                                                                 |
|---------------------|----------------------|---------------------------------------------------------------------------------------|
| Store (website/app) | X-Store-Key header   | Identifies the store, grants only public read + customer features; not a secret login |

| Caller   | Proves identity with   | Notes                                                 |
|----------|------------------------|-------------------------------------------------------|
| Customer | Email + password → JWT | Own cart, profile, orders only                        |
| Admin    | Email + password → JWT | Separate table, separate signing secret, role-checked |

### Password rules

- Hash with bcrypt (cost 12); never store, log or return plain passwords or hashes.
- Minimum length 10; reject very common passwords; same generic message for wrong email or wrong password.
- Rate-limit login endpoints; lock or slow down after repeated failures.

### Tokens

- Access token (JWT): 15 minutes, carries user ID and role only.
- Refresh token: random value, stored hashed in the database, rotated on use, revocable (logout, password change).
- Admin and customer tokens use different secrets and an `aud` claim so one can never work as the other.
- Secrets come from `.env`; if a secret leaks, rotate it and all sessions become invalid.
- Browser clients prefer `HttpOnly`, `Secure`, `SameSite` cookies for refresh tokens; Android stores tokens in encrypted storage.

**Authorization checks (in this order):** valid store key → valid token → account active → role/permission allowed → resource belongs to this user/store.

## 12. Role-Based Access Control

Four admin roles plus customers keep permissions simple; roles are checked by middleware on every admin route, and customers can never write to the catalog.

| Action                                   | customer | order_manager | catalog_manager | admin | super_admin |
|------------------------------------------|----------|---------------|-----------------|-------|-------------|
| Browse published products                | Yes      | Yes           | Yes             | Yes   | Yes         |
| Create/edit/delete products & categories | No       | No            | Yes             | Yes   | Yes         |
| Upload product images                    | No       | No            | Yes             | Yes   | Yes         |
| View all orders / change order status    | No       | Yes           | No              | Yes   | Yes         |
| View own orders, cart, profile           | Yes      | n/a           | n/a             | n/a   | n/a         |

| Action                         | customer | order_manager | catalog_manager | admin | super_admin |
|--------------------------------|----------|---------------|-----------------|-------|-------------|
| View customers list            | No       | Yes           | No              | Yes   | Yes         |
| Manage stores & store API keys | No       | No            | No              | Yes   | Yes         |
| Manage admin users & roles     | No       | No            | No              | No    | Yes         |
| View audit logs                | No       | No            | No              | Yes   | Yes         |

## Rules

- Deny by default: a route without an explicit allowed role returns 403.
- Roles are stored in the database and read from the token; a role change takes effect when the access token expires (15 minutes).
- Ownership check for customers: a customer can only read or change rows where `customer_id` is their own ID.
- The last `super_admin` account cannot be deactivated or demoted.
- Role changes and permission denials are written to the audit log.

## 13. Multi-Store Architecture

One database serves many stores: the API works out which store is calling from its key, then every query is scoped to that store. This is a shared-database, shared-schema design, the simplest model that still isolates stores.

### How a store is identified

1. Each store gets a row in `stores` and a random API key shown once; only its hash is stored in `store_api_keys`.
2. The storefront sends `X-Store-Key` on every request.
3. `storeResolver` middleware hashes the key, finds the active store and puts `req.store` on the request. Unknown, revoked or inactive key → 401.
4. The store ID is never taken from the request body or URL; it always comes from `req.store`.

### What is shared and what is per store

| Shared across stores                                         | Per store (has <code>store_id</code> )                    |
|--------------------------------------------------------------|-----------------------------------------------------------|
| Products, variants, images, categories (the central catalog) | <code>product_stores</code> visibility and price override |
| Admin users and roles                                        | <code>orders</code> , <code>carts</code>                  |

| Shared across stores                           | Per store (has <code>store_id</code> )                                                               |
|------------------------------------------------|------------------------------------------------------------------------------------------------------|
| Inventory (one stock pool, unless split later) | <code>customers</code> registration origin ( <code>store_id</code> ),<br><code>store_api_keys</code> |

## Isolation rules

- Public catalog queries join `product_stores` with `store_id = req.store.id` and `is_visible = 1`; a product not enabled for a store does not exist for it.
- Cart and order reads/writes always include `store_id`; a customer of store A cannot read store B data even with a valid ID.
- CORS allows only the origins registered for that store.
- Admins see all stores; an admin dashboard filters by a chosen store.
- Adding a store = insert a store row, generate a key, enable products, add its origin to CORS. No code change.